Sudoku Solver System

UML Use Case Model and UML Class Diagrams

1. Modified UML Use Case Diagram

The diagram keeps the original system boundary and the two external actors. The main student goal is to solve the five supplied Sudoku puzzles. The main use case includes loading and validating the input files, finding all valid solutions, displaying the puzzle and results, and writing the solutions to output files.

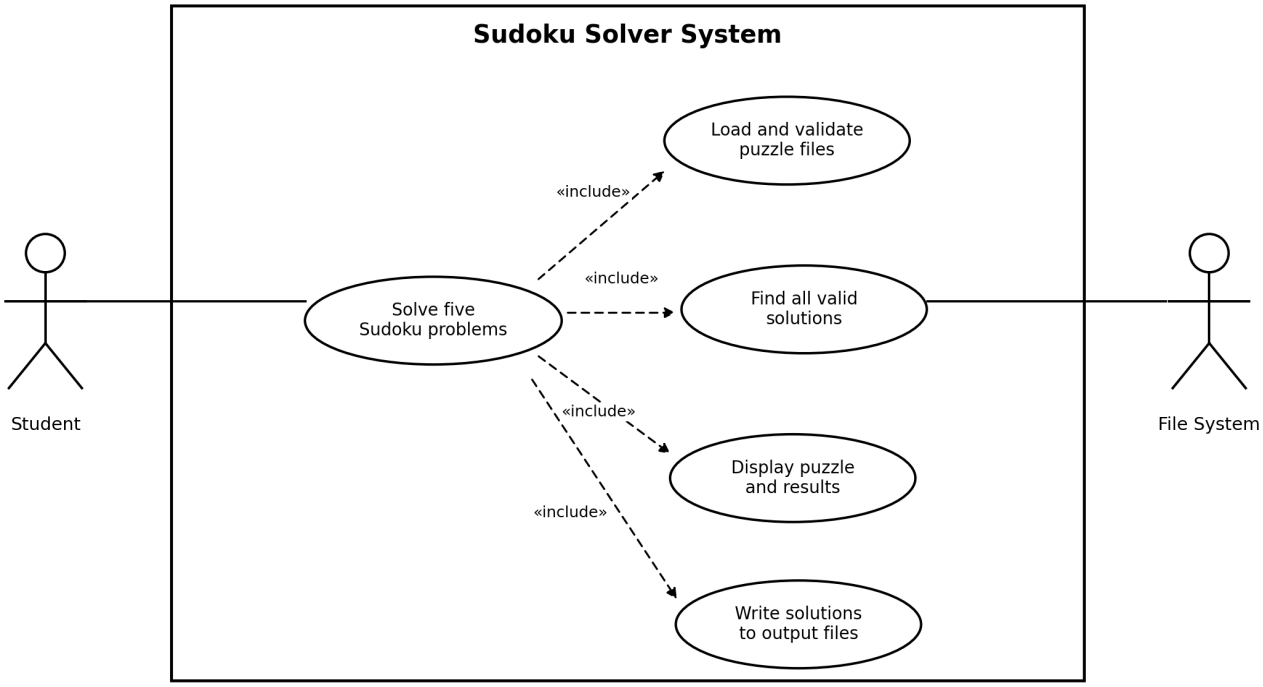


Figure 1. Updated use case diagram for the Sudoku Solver System.

2. Use Case Specification

Use case identification / discovery

Use case name: Solve Five Sudoku Problems
Primary actor: Student
Supporting actor: File System
Goal: Read five Sudoku puzzle files, validate each puzzle, find every valid solution, show the results, and save the solutions to corresponding output files.
Trigger: The student starts the Sudoku Solver program.

Brief description

The student runs the program to solve five Sudoku puzzles stored in text files. For each file, the system loads the 9-by-9 grid and checks whether the starting values follow Sudoku rules. If the puzzle is valid, the solver uses recursive backtracking to search for all possible solutions. The system displays the puzzle and its results, then writes the solutions to output files. If a file cannot be read, the puzzle is invalid, or no solution exists, the system reports the appropriate result.

Step-by-step flow of events

Step	Actor / System	Event
1	Student	Starts the Sudoku Solver program.
2	System	Begins processing the five expected puzzle files (puzzle1.txt through puzzle5.txt).
3	System / File System	Opens the current puzzle file and reads its 9-by-9 grid.
4	System	Checks that the input has the expected format and that the given digits do not violate row, column, or 3-by-3 block rules.
5	System	If valid, searches for solutions using recursive backtracking, trying permitted digits in empty cells.
6	System	Records each complete valid grid found; continues searching until all possibilities have been explored.
7	System	Displays the original puzzle and either all solutions or a 'No solution found' message.
8	System / File System	Writes the solution result to the corresponding output file (for example, solution1.txt).
9	System	Repeats the process for the remaining puzzle files and finishes after all five have been processed.

Detailed use case description

Item	Description
Use case	Solve Five Sudoku Problems
Primary actor	Student
Supporting actor	File System
Preconditions	The program is available to run. The five input files are in the expected location and use the required puzzle format. Each puzzle is
Postconditions — success	The system has displayed the results for each readable and valid puzzle. Every solution found has been recorded, and the corresp
Postconditions — failure	The system reports an input, validation, or no-solution problem for the affected puzzle. It continues with other puzzle files when the
Main success scenario	The student starts the program; the system loads and validates each puzzle; the solver searches for all solutions; the system displa
Alternative flow A — file unavailable	If an input file cannot be opened, the system reports that the file could not be read. It skips that puzzle and proceeds to the next exp
Alternative flow B — malformed input	If the file does not contain a valid 9-by-9 puzzle representation, the system reports invalid input for that file and does not attempt to
Alternative flow C — conflicting clues	If the initial digits repeat in a row, column, or 3-by-3 block, the system identifies the puzzle as invalid and does not search for soluti
Alternative flow D — no solution	If the starting puzzle is valid but the search finds no completed grid, the system displays 'No solution found' and writes that result to
Alternative flow E — multiple solutions	If more than one completed grid satisfies the Sudoku rules, the system records and displays every solution found, rather than stopp
Business rules	A completed Sudoku grid must contain digits 1–9 exactly once in every row, every column, and every 3-by-3 block. Given clues mu

3. UML Class Diagram

The class diagram represents the two classes used by the Sudoku solver: SudokuBoard and SudokuSolver. It shows attribute visibility, attribute types, method visibility, parameter lists, return types, and the ownership relationship between the solver and the board objects.

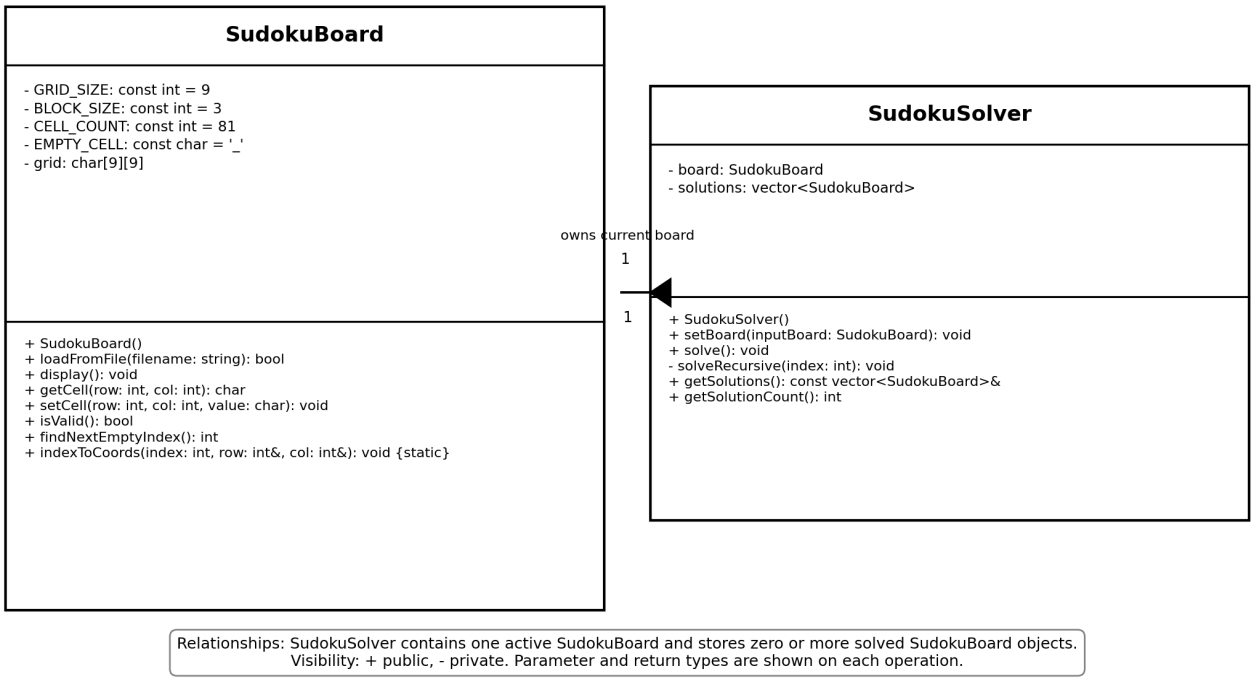


Figure 2. UML class diagram for the Sudoku Solver System.

Class responsibilities and relationships

SudokuBoard: Represents one 9-by-9 Sudoku grid. It stores the cell values and provides operations to load, display, inspect, update, and validate the grid. The static `indexToCoords` helper converts a one-dimensional cell index into a row and column position.

SudokuSolver: Controls the solving process. It holds the current board and stores the completed boards discovered during the search. Its private recursive method performs the backtracking search, while the public methods allow a caller to set a puzzle, start solving, and retrieve the solutions.

Relationship: SudokuSolver has a composition relationship with SudokuBoard: the solver contains its active board and stores solution boards by value in its solutions collection. The solver therefore manages the board objects used during its solving process.

Visibility key

+ public — accessible to other classes or the program's calling code.
– private — accessible only within the class.
Types and signatures are based on the SudokuBoard / SudokuSolver structure used in the assignment. If your final .cpp file uses a slightly different parameter type or const qualifier, update the corresponding UML operation so it matches the submitted source exactly.